

Managing Application Configuration and Secrets

Intermediate Kubernetes — Module 5 of 13

Enterprise EKS Platform





Module Agenda

- The Twelve-Factor App — Config Principle
- ConfigMaps: Creation and Consumption Patterns
- Secrets: Types, Creation, and Consumption
- Environment Variable Precedence
- Volume Projection and subPath Mounts
- Immutable ConfigMaps/Secrets and Encryption at Rest
- External Secrets Operator with Vault



The Twelve-Factor App

Factor III: Store Config in the Environment



Why Externalize Configuration?

Anti-Pattern: Embedded Config

- Config hardcoded in application code
- Requires rebuilding images for each environment
- Secrets committed to source control
- No separation of concerns

Best Practice: External Config

- Config injected at runtime
- Same image across dev, staging, prod
- Secrets managed independently
- Config changes without redeployment

Kubernetes Config Primitives

Kubernetes provides two native resources for external configuration:

- **ConfigMaps** — Non-sensitive configuration data (key-value pairs, files)
- **Secrets** — Sensitive data (credentials, tokens, certificates)



ConfigMaps

Externalizing Non-Sensitive Configuration



Creating and Consuming ConfigMaps

Creation approaches

- **Literals** — individual key-value pairs (simple, unwieldy at scale)
- **Files / directories** — filename becomes the key, contents become the value
- **Declarative YAML** — version-controlled, GitOps-friendly (recommended)

Consumption patterns

- **valueFrom** — map individual keys to env vars
- **envFrom** — bulk-import all keys (optional prefix; skips invalid names)
- **Volume mounts** — each key becomes a file; auto-updates (~60s)



Hot Reload Behavior Deep Dive

How Volume-Mounted ConfigMap Updates Work

- Kubelet periodically syncs ConfigMap data (default: every 60s)
- Updates use atomic symlink swap: `..data` → `..data_tmp` → rename
- All files update atomically together
- Total propagation delay: kubelet sync + API server cache TTL

Kubernetes Secrets

Managing Sensitive Configuration Data

Secret Types in Kubernetes

Type	Description	Use Case
Opaque	Generic, arbitrary user data	Passwords, API keys, tokens
<code>kubernetes.io/tls</code>	TLS cert + private key	Ingress TLS termination
<code>kubernetes.io/dockerconfigjson</code>	Docker registry credentials	Pulling from private registries
<code>kubernetes.io/basic-auth</code>	Basic auth credentials	Username/password pairs
<code>kubernetes.io/ssh-auth</code>	SSH private key	Git clone, SSH access
<code>kubernetes.io/service-account-token</code>	Service account token	API authentication (legacy)



Creating and Consuming Secrets

Creation

- **Literals / files** — CLI leaves credentials in shell history; prefer files or external secrets
- **Declarative YAML** — data (base64) or `stringData` (plain text, converted on storage)

Base64 is NOT encryption — anyone with `get secret` access can decode it instantly.

Consumption

- **secretKeyRef** — inject individual keys as env vars
- **Volume mounts** — expose as files; use `defaultMode: 0400` so only the app can read them



Environment Variable Precedence

Understanding Override Order



Environment Variable Precedence Rules

Order of Precedence (highest to lowest):

1. **env[].value** — Inline value defined directly in the Pod spec
2. **env[].valueFrom** — Single key reference (configMapKeyRef / secretKeyRef)
3. **envFrom** — Bulk import from ConfigMap or Secret

Set defaults via envFrom, then override specifics with explicit env entries. Among multiple envFrom sources, later ones override earlier ones.



Advanced Volume Patterns

Projected Volumes and subPath Mounts

Projected Volumes

Combine multiple sources into one mount

- **configMap** + **secret** — config and credentials together
- **downwardAPI** — expose pod metadata as files
- **serviceAccountToken** — scoped token with audience + expiration
(modern approach, replaces legacy auto-mounted tokens)



subPath Mounts

✗ Without subPath

Mounting at a directory path
replaces the entire directory
— all existing files are hidden.

✓ With subPath

Mount a **single file** into a
directory, preserving the other
existing files alongside it.

Trade-off: subPath mounts do NOT receive automatic updates. For hot reload, use a regular volume mount.

Immutable ConfigMaps and Secrets

Set `immutable: true` on a ConfigMap or Secret — it can no longer be modified, only deleted and recreated.

Benefits

- Protects against accidental changes
- Reduces API server load and improves performance
- Enables versioned config rollout

Update Strategy

- Create new ConfigMap with version suffix
- Update Deployment reference; rolling update applies it
- Delete old ConfigMap after rollout



Secret Best Practices

Security Beyond Base64



RBAC for Secrets

RBAC is your first line of defense

- Restrict reads to **specific resource names** — never grant blanket list/watch on all secrets
- Bind a least-privilege Role to each workload's ServiceAccount

Anyone with pod exec access can read env vars from `/proc` — file-mounted secrets with restricted permissions are safer.

Encryption at Rest

By default, Secrets are stored unencrypted in etcd!

AWS EKS Encryption

- Enable via `secretsEncryption.keyARN` in the cluster config
- Uses AWS KMS envelope encryption

What Encryption at Rest Does

- Encrypts Secret data before writing to etcd
- Decrypts transparently on read via API server
- Protects against etcd backup exposure
- Does NOT protect against API access



Enterprise Secrets with Vault

HashiCorp Vault + External Secrets Operator



Vault Architecture Overview

Core Components

- **Secrets Engines** — KV v2, database, PKI, transit
- **Auth Methods** — Kubernetes, OIDC, LDAP, AppRole
- **Policies** — path-based ACLs in HCL (deny-by-default)
- **Audit Devices** — every access logged

Why Vault for K8s?

- Centralized secret management across clusters
- Dynamic, short-lived credentials
- Full audit trail of secret access
- Versioned secrets with rollback

KV v2 is the most common engine — stores versioned key-value pairs at paths like `apps/myapp/db`



Vault + Kubernetes Auth

How Pods Authenticate to Vault

Pods use their ServiceAccount JWT token. Vault validates it against the Kubernetes API server and issues a scoped Vault token. A Vault role binds a ServiceAccount + namespace to policies, with a TTL controlling token lifetime.

Flow: Pod SA token → Vault validates with K8s API → issues Vault token with bound policies → Pod reads secrets



External Secrets Operator Pattern

ESO Components

- **SecretStore** — connection config to Vault (use `ClusterSecretStore` for shared infra)
- **ExternalSecret** — declares what to sync and from which remote paths
- **Operator** — reconciles on `refreshInterval`, creating a native K8s Secret

Workloads consume standard K8s Secrets — no code changes needed.



Secret Rotation Workflow

End-to-End Rotation Flow

1. **Update in Vault** — rotate credential via CLI, API, or pipeline
2. **ESO detects change** — next `refreshInterval` polls Vault
3. **K8s Secret updated** — ESO patches the native Secret
4. **Pod consumption** — volume mounts auto-update (~60s); env vars need restart

Tip: Use `stakater/Reloader` to trigger rolling restarts when Secrets change — essential for env-var consumers.

Vault + ESO Best Practices

Security

- NetworkPolicy: restrict ESO egress to Vault only
- Minimum RBAC for ESO service account
- Enable Vault audit logging
- Encrypt etcd at rest (AWS KMS)

Lifecycle

- deletionPolicy: Retain for critical secrets
- Size refreshInterval for freshness vs API cost
- Label ESO-managed Secrets for audit trail

Monitoring

- Alert on ExternalSecret non-Ready status
- Track `externalsecret_sync_calls_error` metric
- Monitor provider API rate limits



Lab 5: ConfigMaps and Secrets

Hands-On Exercise

- Create ConfigMaps using literal and file approaches
- Consume ConfigMaps as environment variables and volume mounts
- Create and consume Secrets of type Opaque and TLS
- Observe hot reload behavior with volume-mounted ConfigMaps
- Use projected volumes to combine ConfigMap + Secret + Downward API
- Store secrets in HashiCorp Vault and sync via External Secrets Operator

Duration: ~30-40 minutes

Key Takeaways

1. Separate config from code — Use ConfigMaps and Secrets to make your images environment-agnostic.

2. Volume mounts > env vars — Volumes support hot reload (~60s) and are more secure for secrets.

3. Base64 ≠ encryption — Enable encryption at rest (AWS KMS) and use ESO + Vault for real workloads.

4. Use immutable resources — Prevent accidental changes and reduce API server load.

? Questions & Discussion

Module 5: Managing Application Configuration and Secrets

Up Next: Module 6 — Accessing Your Applications from Outside — Ingress & Egress